

Energy-Efficient Computation Offloading With DVFS Using Deep Reinforcement Learning for Time-Critical IoT Applications in Edge Computing

Saroj Kumar Panda¹, Man Lin², *Senior Member, IEEE*, and Ti Zhou³

Abstract—Internet of Things (IoT) is a technology that allows ordinary physical devices to collect, process, and share data with other physical devices and systems over the Internet. It provides pervasively connected infrastructures to support innovative applications and services that can automate otherwise intensely laborious manual effort. Edge computing (EC) complements the powerful centralized cloud servers by providing powerful computation capability close to the data source, minimizing communication latency, and securing data privacy. The energy consumption problem has continued to receive much attention from the IoT community in applying various techniques to reduce energy consumption while still meeting the computational demand. In this article, we propose an application-deadline-aware data offloading scheme using deep reinforcement learning and dynamic voltage and frequency scaling (DVFS) in an EC environment to reduce the energy consumption of IoT devices. The proposed scheme learns the optimal data distribution policies and local computation DVFS frequency scaling by interacting with the system environment and learning the behavior of the device, network, and edge servers. The proposed scheme was tested on multiple EC environments with different IoT devices. Experimental results show that this scheme can reduce energy consumption while achieving the IoT application and services timing and computational goals. The proposed scheme has substantial energy savings when compared with the native Linux governors.

Index Terms—Deep reinforcement learning (DRL), dynamic voltage and frequency scaling, edge computing (EC), edge server, energy consumption, Internet of Things (IoT), offloading.

I. INTRODUCTION

INTERNET of Things (IoT) is a technology that allows ordinary physical devices to collect, process, and share data with other physical devices and systems over the Internet. IoT is changing the way these devices can interact with the environment around them to improve the quality of day-to-day life through many smart systems, such as smart cities, smart health care, smart homes, etc. [22], [26]. Traditionally IoT devices offload the data to centralized cloud servers, which have an abundance of computational resources available. The

cloud servers perform the computation and send back the results to the IoT devices to take appropriate actions. This approach suffers from several shortcomings, such as network latency, higher network bandwidth requirement, privacy, and security concerns. To address these challenges, edge computing (EC) [31] moves the computation close to the data source, i.e., edge of the network. Edge servers are placed in close proximity to IoT edge devices. The edge devices can offload data to edge servers using various communication mediums, such as Bluetooth, local area network (LAN), etc. This prevents the data from traversing through any public domain, thereby preserving privacy and reducing the risk of security threats to the data. The computation results from edge servers become available to edge devices faster, enabling them to provide a real-time response experience to the end user.

With the evolution in the development of compact IoT devices, today's IoT devices have the computational power and memory to store data, enabling EC to move the computation directly to the data source. However, compared with desktops or servers, IoT devices have limited computation capability. In addition, since IoT devices are deployed in the field, they are normally powered by built-in batteries. Their lifetime, therefore, depends on the computation intensity of the executed applications. With these limitations, running computation-intensive or delay-sensitive applications only on the IoT devices may not meet the application requirement and may reduce the battery lifetime.

Offloading is considered as one of the solutions to the above problems, which transfers some of the computation to other nearby systems, such as edge servers, fog access points (F-APs), etc. A variety of different offloading schemes have been proposed [9], [16]–[18], [30]. These schemes take advantage of high bandwidth local communication mediums and close proximity resource-rich servers to offload computation, enabling the delay-sensitive applications to complete on time and increase IoT device battery life. But, the core challenge lies in how to jointly optimize computation, communication, and energy consumption, and make a controlled computation offloading decision, which is affected by many factors, such as application computation characteristics, communication medium condition, offloading energy consumption, and edge server computing resource utilization. If an offloading decision does not consider the variations in these factors, it

Manuscript received 14 October 2021; revised 12 January 2022; accepted 6 February 2022. Date of publication 23 February 2022; date of current version 7 April 2023. This work was supported by the Natural Sciences and Engineering Research Council of Canada. (Corresponding author: Man Lin.) The authors are with the Department of Computer Science, St. Francis Xavier University, Antigonish, NS B2G 2W5, Canada (e-mail: mlin@stfx.ca). Digital Object Identifier 10.1109/IIOT.2022.3153399

will lead to poor performance. For example, poor communication network conditions or heavily used edge server computing resources or both will hurt the benefits of offloading and may lead to more energy consumption and computation time. In this scenario, performing more computation on the local IoT device is a better option.

Deep learning (DL) using deep neural networks (DNNs) has become an emerging technology for IoT applications and systems [15], [21] to extract meaningful information from the application data and assist in taking intelligent actions. One advantage of DL is that these DL models can be trained offline using high computational resources, such as cloud servers or edge servers and deployed online on IoT devices to perform inference on real-time application data. This is based on the fact that DNN requires less computational resources during inference than in the training phase. Another advantage of DL is that these DL models are highly data parallel, i.e., inference can be done in parallel with multiple instances of the DL model deployed on different EC devices for separate sets of data. We take advantage of this data parallelism to perform a simpler computation offload by offloading a part of the data from the IoT device to nearby edge servers. Both the IoT device and the edge server execute the same computation task on separate sets of data.

In recent years, many reinforcement learning (RL)-based offloading schemes have been proposed [9], [20], where IoT devices learn the best offloading policy by interacting with the application environment. Instead of solving a complex problem of what to offload, IoT devices adjust the offloading policy continuously in a trial-and-error manner by interacting with the system till the best policy is found without the need for prior system knowledge, which is often unavailable in practice. Another effective approach to reduce energy consumption is dynamic voltage and frequency scaling (DVFS), which is one of the most promising solutions to optimize processor energy consumption and control system temperature where the processor frequency and voltage are adjusted to lower levels to reduce power consumption while still meeting the task completion deadlines [2]. Most of the modern processors and operating systems support DVFS.

In this article, we present a DVFS-aware deep reinforcement learning (DRL)-based controlled data offloading (DRLDO) scheme in an EC environment that jointly learns the optimal computation and communication characteristics of the whole system to control the computation speed of the local IoT device and minimize energy consumption while still satisfying the computation time constraints of the IoT application. The contributions of this article are as follows.

- 1) A DVFS-aware DRL-based offloading scheme that minimizes IoT device energy consumption by jointly optimizing partial data offloading ratio, and IoT device CPU frequency based on the current network, edge server states, and the IoT application deadline.
- 2) A new method of measuring energy consumption of IoT devices, such as Raspberry Pi [6] and Nvidia Jetson Nano [4].

- 3) Comparison of results of this scheme using various devices as IoT devices, such as Raspberry Pi, Nvidia Jetson Nano, Linux laptop, etc.

The proposed scheme can be applied to various IoT applications, such as intrusion detection at remote areas [1], [28], where the video frames captured by the cameras can be distributed between the IoT edge and the edge server to detect intruders in a timely and energy-efficient way; manufacturing defect detection in an Industrial IoT (IIoT) settings [24], where measurements collected via distributed sensor networks on the production line can be distributed among IIoT edge devices and edge servers to detect manufacturing defects in a timely and energy-efficient manner helping the manufacturer in reducing the operational costs associated with defective components.

The remainder of this article is organized as follows. Related work is discussed in Section II. Section III outlines the system model, which consists of *Communication Model*, *Offloading Model*, and *Energy Consumption Model*. DRL-based data offloading is introduced in Section IV where we first introduce the concept of DRL, then provide details of the system design, our new method of measuring the energy consumption of IoT devices, such as Raspberry Pi [6] and Nvidia Jetson Nano [4], and describe the details of the proposed DRLDO algorithm. The experimental environment setup and results are discussed in Section V. Finally, we summarize and conclude this article in Section VI.

II. RELATED WORK

Many IoT applications require real-time responses and low energy consumption. In recent years, EC and DVFS have been gaining popularity, with EC providing real-time responses to IoT applications and DVFS saving energy.

A. Learning-Based DVFS

Generally, reducing energy consumption using the DVFS technique highly depends on selecting the right CPU core voltage and frequency according to the varying system states. To reduce energy consumption, many reinforcement learning-based methods have been proposed to learn how to switch DVFS dynamically to adapt to different states. There are extensive researches in the literature that studies DVFS. Recent works focus on intelligently scaling the processor frequencies dynamically to reduce the energy consumption of various systems. Islam and Lin [12] proposed an RL-based DVFS algorithm selection approach that integrates a set of DVFS algorithms and Q -learning to reduce real-time systems energy consumption named hybrid DVFS scheduling. Zhang *et al.* [34] proposed a DRL-based DVFS technique selection scheme for energy-efficient edge scheduling on edge devices. Huang *et al.* [10] proposed a double- Q -learning method for DVFS scaling with the double- Q -table learned through Q -learning. Park *et al.* [25] presented a technique that uses a tree-based piecewise model to build an intelligent CPU-GPU governor to save energy for mobile games. Zhou and Lin [36] presented an intelligent deep-recurrent- Q -network (DRQN) governor that performs DVFS

for dedicated applications with deadline requirements running on cyber-physical-system devices. Biswas *et al.* [3] proposed a method to select appropriate voltage frequency based on workload prediction for many-core architecture. Ramegowda and Lin [29] showed that a hybrid learning-based DVFS methodology can be adapted to various Linux embedded platforms with ARM and Intel architectures when running with mixed (periodic and aperiodic) task sets.

However, as the data generated by IoT edge devices grow extensively and more restricted time constraints for the IoT applications, it is imperative to offload some of the data or computational tasks or both to the edge servers. Thus, different from the above works on DVFS, this article considers resource-constrained systems that cannot finish processing all the data within the deadline (often occurring in many IoT devices supported by EC).

B. Offloading in EC

Many energy-efficient edge task scheduling and computation offloading approaches have been proposed.

Li *et al.* [16] proposed a DRL offloading approach to jointly optimize the offloading decision and resource allocations for a multiuser mobile EC (MEC) system, where multiple user equipment (UE) can perform computation offloading via wireless channels to a MEC server to minimize the sum of delay cost and energy consumption for all UEs.

Huang *et al.* [9] proposed a real-time reinforcement learning (double Q -learning)-based offloading scheme, RRLO, that jointly optimizes task scheduling and offloading for real-time applications in a MEC environment. RRLO offloads a part of the computational tasks to MEC servers and selects the proper DVFS scheme dynamically for the tasks executed locally to minimize the energy consumption of the mobile device.

Zhang *et al.* [35] proposed a DRL-based data and task offloading where the agent chooses a near-optimal solution to adjust the workload and data partitions, intelligent reflecting surface (IRS) parameters, and the time allocation in dynamically changing wireless channel conditions and random arrival of task workload in a MEC environment to jointly optimize the energy efficiency, time latency, and price of offloading computations. IRS is a revolutionizing technology that significantly improves the performance of wireless communication systems. The authors assume that a task can be divided into two parts, one part executed locally and the other on the MEC server in parallel.

Gong *et al.* [8] proposed a DRL-based hybrid offloading model that exploits data offloading over active RF communications and low-power backscatter communications between IoT devices and MEC servers based on network conditions. They maximize the MEC performance through the hybrid offloading and verify that the hybrid offloading, which learns the optimal workload allocation among local computation, active and passive data offloading can boost the energy efficiency by 20 percent when compared with existing offloading schemes.

Lin *et al.* [17] proposed an algorithm for the mobile cloud computing (MCC) task scheduling to minimize the total energy consumption of an application in a mobile device under

the hard constraint of application completion time by representing the task schedule as a directed acyclic graph (DAG). They follow a multistep approach. In the first step, they generate a minimal-delay task scheduling graph. In the second step, energy reduction is performed by migrating tasks toward the cloud or other local cores available that can bring energy reduction without violating the application completion time constraint. They apply the DVFS technique to further reduce energy consumption in the third step.

Ren *et al.* [30] proposed a DRL-based scheme to minimize long-term system energy consumption by offloading computation to F-APs and to the cloud in a multiple IIoT devices scenario. They create a DRL model for each IIoT device that identifies its computation offload serving F-AP based on network and device states. To scale for any growing computation demand, they execute a low complexity greedy algorithm at each F-AP to determine which offloading requests can further, be forwarded to the cloud.

Most of the proposed offloading schemes above offload the computation by offloading tasks to servers, which itself requires computation to prepare the task precedence schedule and decision of task allocation. In this work, we propose a simpler computation offloading scheme by offloading only partial data required for computation to the edge servers with the same computation tasks available at both the IoT device and edge servers. We use trained DL models as IoT applications to validate the scheme.

Another difference is that the proposed works above use formula-based energy consumption estimations and simulation results to validate their proposed scheme and often only estimate processor energy consumption, which may differ significantly from the system energy consumption of the IoT device. Apart from the processor energy, other system components, such as memory, sensors, etc., also consume energy as well as running other software components on the system. In our proposed scheme, we use real devices and optimize the whole device system energy.

III. SYSTEM MODEL

In this section, we present the details of the communication model, offloading model, and energy consumption model adopted in the system under investigation.

A. Communication Model

Generally, IoT devices and edge servers are connected to the Internet either through wireless mode (Wi-Fi) or through the physical mode (Ethernet). The IoT device offloads the data to the edge server using TCP/IP over the LAN. However, it is also possible to use other communication mediums, such as Bluetooth, Wi-Fi Direct, etc. The data transmission time (U_t) depends on the communication medium data transmission rate (r_u) and the number of bits in the data (d_n), as given in the following equation:

$$U_t = \frac{d_n}{r_u}. \quad (1)$$

The communication medium's data transmission rate may vary over time depending on the communication medium state.

Our proposed scheme takes into account the variation of data rates while selecting the best policy.

B. Offloading Model

The data d can be divided into two parts (2), where d_e is the data processed by the IoT device, and d_s is the data processed by the edge server

$$d = d_e + d_s. \quad (2)$$

If the data are offloaded to multiple servers, then d_s will be the sum of the data that is offloaded to all the servers (3), where d_{s_i} is the data offloaded to the i th server and n is the number of edge servers used

$$d_s = \sum_{i=1}^n d_{s_i}. \quad (3)$$

Let T_{s_i} be the processing time of the i th server for the data d_{s_i} . Using (1) the response time of the i th server (R_{s_i}) will be as follows:

$$R_{s_i} = U_{t_i} + T_{s_i} \quad (4)$$

where U_{t_i} is the transmission time of the data d_{s_i} sent to the i th edge server.

Let T_e be the processing time of the data d_e processed locally, and the response time R of the application can now be computed as maximum of the local processing time and response times of all the servers

$$R = \max [T_e, R_{s_1}, R_{s_2} \cdots R_{s_n}] \quad (5)$$

where $R_{s_1}, R_{s_2}, \dots, R_{s_n}$ are response times of respective edge servers.

If T is the expected response time of the application for the data d , the goal of offloading is to satisfy the following condition:

$$\max [T_e, R_{s_1}, R_{s_2} \cdots R_{s_n}] \leq T. \quad (6)$$

C. Energy Consumption Model

Our proposed approach aims to minimize the total energy consumption of the IoT device, which consists of two parts: 1) dynamic energy consumption and 2) static energy consumption. The static energy consumption is due to the leakage current that happens continuously even if the CPU is idle. The dynamic energy is consumed for computation and communication. Computation energy depends on the CPU core voltage (V), frequency (f), and execution time (t) (8). The communication energy depends on the amount of data transferred (d_n), transmission rate (r_u), and the transmission power (q_t) as given in (7)

$$E_t = \frac{d_n \cdot q_t}{r_u} \quad (7)$$

$$E_c \propto V^2 \cdot f \cdot t. \quad (8)$$

The total energy consumed by the application task (E) can be calculated by the following equation:

$$E = E_c + E_l + E_t \quad (9)$$

where E_c , E_l , and E_t are the energy consumed by the IoT device for computation, leakage current, and communication, respectively. From (6) and (9), the energy optimization problem can be formulated as follows:

$$\begin{aligned} &\text{Minimize } (E_c + E_l + E_t) \\ &\text{s.t. } \max [T_e, R_{s_1}, R_{s_2} \cdots R_{s_n}] \leq T. \end{aligned} \quad (10)$$

IoT device local execution time and computation energy both depend on the CPU frequency and the amount of data processed locally. The response times of servers and communication energy depend on the data distributed to the servers, network conditions, and server resource availability. Therefore, finding an appropriate data offloading policy and local CPU frequency based on the current network and server state is critical to meet the application time constraint and energy optimization.

IV. DEEP REINFORCEMENT LEARNING-BASED DATA OFFLOADING

In this section, we first present a brief introduction of DRL in general. Then, we discuss the steps involved in the proposed offloading scheme DRLDO.

A. Deep Reinforcement Learning

Reinforcement learning is an autonomous learning process where the learning agent learns the optimal policy through knowledge obtained by trial and error and continuously interacting with a dynamic environment [27]. The agent periodically takes actions in the environment and gradually improves the strategy of action selection by processing the feedback received from the environment until the optimal policy is achieved. In each period, the agent first observes the environment state s and then takes action a on the environment. For the action, the agent receives either a reward r or penalty p as the feedback from the environment, and the environment moves to a different state s' for the next period. The agent repeats the above process to learn the optimal policy π that optimizes the long-term accumulative reward.

Q -learning is one of the simplest model-free RL algorithms that has been applied to various domains and applications [12], [13]. It learns the Q -value $Q(s, a)$ of any action in any particular state by taking that action. The iterative update of the Q -values is continued till the changes to Q -values fall below a threshold value where the Q -table converges and the Q -learning algorithm training can be stopped. However, for Q -learning to work the state-action space should be small, finite, and Q -values for each state-action pair should be computed. But, in many practical applications, the state or action space is huge. Storing all possible states, actions, and their corresponding Q -values would take up a huge amount of memory. Training the Q -table in such scenarios to convergence would also take a vast amount of time. We use the application deadline and the edge server response time as the state parameter in our *System Design*. The edge server response time will have a wider range of possible values depending on the network and server loads. Therefore, instead of computing and storing each state-action pair Q -values, we use a DNN to estimate

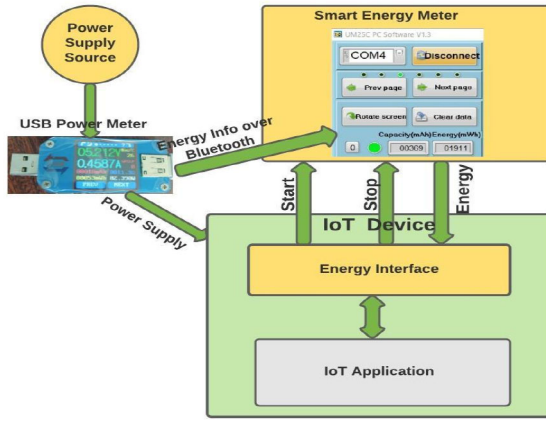


Fig. 2. System architecture of IoT device energy measurement.

limit (RAPL) [5] when we use Linux laptop with Intel CPU [11] as the IoT device. We use the Linux Power Capping Framework that provides the *sysfs* interface to directly read the energy counters to measure the energy consumed by the whole system, including the memory and the System on Chip (SoC). For example, reading the file content `/sys/class/powercap/intel-rapl/intel-rapl:0/energy_uj` provides the energy consumed by package-0, which includes energy consumption for all the CPU cores, DRAM, and CPU uncore. Similarly reading the file content `/sys/class/powercap/intel-rapl/intel-rapl:1/energy_uj` provides the energy consumed by the SOC. We use the sum of package-0 and the SoC energy as the total system energy for the Linux laptop.

- 8) *Frequency Controller*: This component is responsible for setting the IoT device CPU frequency to the action frequency.
- 9) *App Core Engine*: This is the central unit of the proposed system, which trains and maintains the trained DRL model, launches the IoT application, and controls all other components of the system.
- 10) *Offload Controller*: This component offloads the data to edge servers as per the distribution ratio of the action being taken by *App Core Engine*.
- 11) *App Data*: This component holds the data to be processed by the IoT application.

C. Energy Consumption Measurement

We use an external device USB Power Meter, UM25C [19], to measure the system energy of Raspberry Pi 4 Model B [6] and Nvidia Jetson Nano 2 GB [4]. The system architecture of the connection and communication is schematically illustrated in Fig. 2 [23, p. 7]. The IoT device is powered through the power meter and the power meter sends the instantaneous energy consumption information to its Windows Bluetooth application to display it on the screen. We developed two software components: 1) an *Energy Interface* and 2) a *Smart Energy Meter* application to read the energy from the screen. Using the *Energy Interface*, the IoT application communicates with the *Smart Energy Meter* background application, which captures the Windows machine screen and processes

Algorithm 1 Proposed DRLDO Algorithm

```

1: Set list of application deadlines
2: Set list of actions (frequency and ratio combination)
3: Set number of iterations
4: Set explore/exploit parameter
5: Set penalty = Any higher number than the expected energy consumption
6: for number of iterations do
7:   for each application deadline do
8:     Determine the system state
9:     if iteration within explore then
10:      Select a random action
11:    else
12:      Select the action with min estimated action
      value from the DRL model given the state
13:    end if
14:    Set local CPU frequency as in selected action
15:    Perform computation and communication as per
    selected action data distribution
16:    Get completion time and energy consumption
17:    if completion time > application deadline then
18:      Set reward = penalty
19:    else
20:      Set reward = energy consumption
21:    end if
22:    Train the DRL model with the data collected
23:  end for
24: end for
25: Save the DRL Model

```

the captured screens through computer vision algorithms to extract the energy information and sends it back to the IoT application through the *Energy Interface*.

D. Proposed DRLDO Algorithm

Algorithm 1 depicts the proposed DRLDO algorithm. Application deadlines are set by the user. A set of CPU frequencies and data distribution combinations are defined as the actions possible on the system. Initially, the application sends some data to the edge servers for processing through the *Network Interface* and observes the response times, which captures the network and server behavior at that time point. These response times and the application deadline determine the system state. The system deploys an explore and exploit strategy in the training phase. The explore/exploit parameter defines the percentage of the number of iterations the DRL agent would explore. It initially explores the environment by selecting random action and then exploits by selecting the action with the minimum DRL model estimated action value in the observed state. A part of the data is offloaded to the edge server as per the selected action data distribution, and the other part is processed locally in parallel after setting the CPU frequency to the desired frequency. After the action is completed as determined by (5), the *App Core Engine* gets the energy consumption information as the reward and trains the model with the new information collected. The new state is

determined from the response times of the edge servers for the last action. We use min estimated action values as our goal is to minimize energy consumption. A penalty of a higher number is used as a reward if the computation completion time exceeds the application deadline. The new state observed may be the same as the previous state if there is no change in the network or server processing capability and would be different if there is any change in the network condition or the server processing capability. The process of observing the state and executing an action in the observed state is repeated and the trained DRL model is saved after the iteration for future use. The time complexity of the algorithm is $O(N * D)$, where N is the number of iterations and D is the number of deadlines.

V. EXPERIMENT SETUP AND RESULTS

In this section, we present the details of the experimental setup, the results of the proposed scheme on a set of IoT devices, and an analysis of their results. We also present a comparison of the energy consumption by the proposed method versus the Linux native ondemand and conservative governors.

We used MNIST [32] classification as the IoT application, developed using TensorFlow 2.2 in Python 3.7. MNIST is a data set of handwritten digits 0–9 with 60 000 training and 10 000 testing images. After training on the training images, the trained model is saved and then deployed on both the IoT device and on the edge server. We use UM25C [19], USB Power Meter to measure energy consumption when using Raspberry Pi [6] or Jetson Nano [4] as the IoT device. As milliWatt hours (mWh) is the unit of energy measured by the power meter, we use 100 000 MNIST test images in our experiment by classifying the original 10 000 test images 10 times through the MNIST model to observe noticeable differences between the energy consumption readings for the various distribution of the data between the IoT device and the edge server.

We use one IoT device (user) and one edge server in our experiment. We define a set of actions consisting of the IoT device-supported CPU frequencies and data distribution ratios between the IoT device and the edge server. The offloading model framework can be extended to find optimal data distribution for multiple IoT devices (multiuser) with multiple edge servers available where each IoT device (user) adds data distribution ratios for each edge server to which it would offload data and its own ratio to the action parameters such that the sum of the data distributions of those actions adds up to 100. Similarly, each IoT device will add the response time from each server to its state parameters. The IoT devices (users), however, will act independently of each other while sharing common edge servers. We define one or more application deadlines. To measure the server response time initially, we send 10 000 MNIST images to the edge server for classification. After receiving the results, we compute per image edge server processing time as the response time. The combination of response time and the application deadline determines the state. After the initial state is determined, we take random action in that state and measure the energy consumption for the action. If the action gets completed within the application

deadline, then the energy consumption is used as the action value; otherwise, a penalty of a high number is used as the action value. Subsequent states are determined by observing the response time for the action taken in the previous step. We repeat the steps of determining the state, taking an action in the state, measuring the reward, and training a DRL model as explained in Algorithm 1. The trained DRL model is saved after each training to be used in the subsequent training. We use TCP/IP over the LAN as the communication medium to offload data from the IoT edge to the edge server. The DRL model is trained at various times of the day to capture the variations in the network. We apply intermittent CPU load on the edge server by running other CPU-intensive applications on the edge server to create intermittent CPU load. Both the variations in the network and the edge server load affect the response time from the edge server, and we capture that in the state with response time as one of the state parameters. By using different devices, such as Jetson Nano 2 GB [4], Raspberry Pi 4 [6], and Linux laptop with Intel CPU [11] as the IoT devices, we also capture the impact of local system configuration changes on the experimental results.

To validate our scheme, we deploy the trained DRL model on the IoT device. We follow the approach described above to find the initial server response time and determine the system state. We feed the state to our trained DRL model to estimate the action values for the actions and find the action with the minimum estimated action value, perform the action, and measure the energy and time consumption. Subsequently, we take the rest of the actions available in that state one after another and measure the corresponding energy and time consumption. We compare the results obtained for each of those actions.

A. Linux Laptop

The Linux laptop used as the IoT device has Intel Core i3-6006U Processor [11], which supports 16 frequencies: 2 000 000, 1 900 000, 1 800 000, 1 700 000, 1 600 000, 1 500 000, 1 400 000, 1 300 000, 1 100 000, 1 000 000, 900 000, 800 000, 700 000, 600 000, 500 000, and 400 000 measured in unit kHz. The machine supports DVFS with separate voltage for most of the frequencies as given in Table I. The mapping was obtained by changing the CPU core frequency and observing the corresponding voltage through the open-source tool *CPU-X* [33].

CPU-X is a free and open-source software that gathers information on CPU, motherboard, and much more, including the CPU core voltage for systems running GNU/Linux and FreeBSD. It uses *libcpuid* [7], a C library for x86 CPU detection and feature extraction to get the relevant information. Currently, *libcpuid* does not have support for ARM CPU-based systems, which follow reduced instruction set computing (RISC) architecture.

We selected three frequencies [800 000, 1 400 000, 2 000 000] for the local computation and data distribution ratios as [30, 50, 70], which generates nine actions for the environment as [(800 000, 30), (800 000, 50), (800 000, 70), (1 400 000, 30), (1 400 000, 50), (1 400 000, 70), (2 000 000, 30), (2 000 000, 50), (2 000 000, 70)]. We used Legion 5i

TABLE I
INTEL CORE I3 PROCESSOR FREQUENCY TO VOLTAGE MAPPING

Frequency(MHz)	Voltage Levels(V)
400	0.600
500	0.600
600	0.600, 0.610
700	0.610, 0.620
800	0.620
900	0.630
1000	0.640, 0.650
1100	0.660, 0.670
1300	0.690, 0.700
1400	0.700, 0.710
1500	0.720
1600	0.730, 0.740
1700	0.760, 0.770
1800	0.780, 0.790
1900	0.800, 0.810
2000	0.820, 0.830

TABLE II
LINUX TEST RESULTS FOR ALL ACTIONS WITH DEADLINE 40 000 MS

Action	Energy/Penalty	Deadline exceed?
(800000, 30)	24.45	No
(800000, 50)	25.88	No
(800000, 70)	28.87	No
(1400000, 30)	25.94	No
(1400000, 50)	27.60	No
(1400000, 70)	29.86	No
(2000000, 30)	27.61	No
(2000000, 50)	30.69	No
(2000000, 70)	33.18	No

as the edge server, 40 000 and 20 000 ms as two sets of application deadline and 50.0 as the penalty action value whenever the action taken does not meet the application deadline. We use Lenovo Legion 5i laptop (Legion 5i) [14] running Windows 10 as the edge server. We use 50.0 as the penalty whenever any action does not meet the application deadline.

On testing with the application deadline set to 40 000 ms, we obtained response time as 5 ms and the state as (5, 40 000). Passing this state to the trained DRL model, the estimated action values for the above nine actions were [26.33 27.39 31.24 27.96 26.51 29.32 27.47 29.97 33.55] with minimum value as 26.33, which corresponds to index 0 with the action (800 000, 30) and that becomes the best action for that state. The results for the best action and other actions are given in Table II. As seen from the table, the best action (800 000, 30) meets the application deadline and has the lowest energy consumption of 24.45 mWh. The rest of the actions have higher energy consumption than 24.45 mWh.

On testing with the application deadline set to 20 100 ms, a slight variation of 100 ms from the deadline 20 000 ms, which the DRL agent had seen during training, we observed server response time as 5 ms and hence the state as (5, 20 100). Passing this state to the trained DRL model, the estimated action values for the above nine actions were [53.58 49.75 55.27 51.34 18.10 20.84 19.60 22.16 27.13] with minimum value as 18.10, which corresponds to index 4 with the action (1 400 000, 50) and that becomes the best action for that state. The results for all the actions are given in Table III. As seen

TABLE III
LINUX TEST RESULTS FOR ALL ACTIONS WITH DEADLINE 20 100 MS

Action	Energy/Penalty	Deadline exceed?
(800000, 30)	50.00	Yes
(800000, 50)	50.00	Yes
(800000, 70)	50.00	Yes
(1400000, 30)	50.00	Yes
(1400000, 50)	19.83	No
(1400000, 70)	22.40	No
(2000000, 30)	50.00	Yes
(2000000, 50)	22.89	No
(2000000, 70)	26.59	No

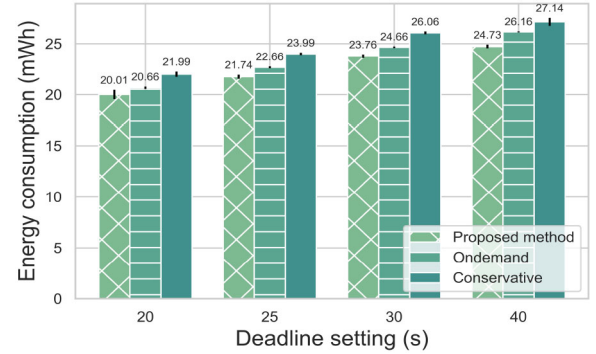


Fig. 3. Linux laptop energy consumption comparison.

from the table, the best action (1 400 000, 50) meets the application deadline and has the lowest energy consumption of 19.83 mWh. Other actions either miss the deadline and receive a penalty of 50.0 or meet the deadline but have a higher energy consumption than 19.83 mWh. The action (800 000, 30), which was the best action for the 40 000 ms deadline is no longer the best action for the 20 100 ms deadline as it misses the deadline for running the computation on a lower frequency. The trained DRL model is able to estimate the action values correctly for some variations in the observed state even if it had not seen that state during training.

We compared energy consumption from the proposed method with the Linux governors *ondemand* and *conservative* where we use the same data distribution ratio found by the proposed method and leave the CPU frequency scaling to the governors. Both *ondemand* and *conservative* governors change the frequency based on the CPU utilization where *conservative* governor scales the frequency up or down gradually whereas *ondemand* can scale up to highest frequency or scale down to lowest frequency more rapidly. The comparison results for various deadlines of 20, 25, 30, and 40 s are given in Fig. 3. The proposed method has the lowest energy consumption for all the deadline settings. The *ondemand* governor consumes more energy in the range of 3.25%–5.78% when compared to the proposed method energy consumption. Similarly, *conservative* governor consumes more energy in the range of 9.68%–10.35% when compared to the proposed method energy consumption as shown in Fig. 4.

B. Nvidia Jetson Nano 2 GB (Nano 2 GB)

Nano 2 GB [4] is a small and powerful board designed for AI and robotics applications. It supports 15 CPU

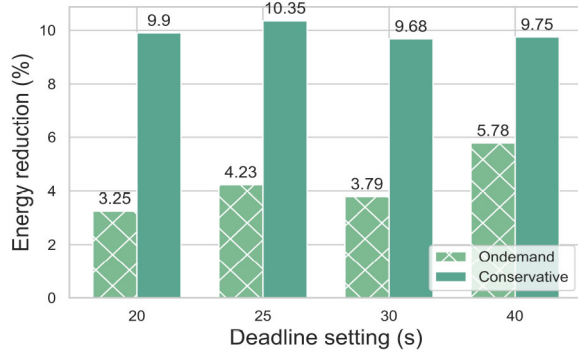


Fig. 4. Proposed method energy reduction compared to native governors on Linux laptop.

TABLE IV
JETSON NANO 2 GB TEST RESULTS FOR ALL
ACTIONS WITH DEADLINE 80 000 MS

Action	Energy/Penalty	Deadline exceed?
(307200, 30)	33.0	No
(307200, 50)	99.0	Yes
(307200, 70)	99.0	Yes
(518400, 30)	41.0	No
(518400, 50)	41.0	No
(518400, 70)	99.0	Yes
(1479000, 30)	36.0	No
(1479000, 50)	38.0	No
(1479000, 70)	41.0	No

frequencies: 102 000, 204 000, 307 200, 403 200, 518 400, 614 400, 710 400, 825 600, 921 600, 1 036 800, 1 132 800, 1 224 000, 1 326 000, 1 428 000, and 1 479 000 measured in unit kHz. We selected three frequencies [307 200, 518 400, 1 479 000] for the local computation and data distribution ratios as 30, 50, and 70, which generates the action space consisting of nine actions [(307 200, 30), (307 200, 50), (307 200, 70), (518 400, 30), (518 400, 50), (518 400, 70)].

We used Lenovo Legion 5i laptop (Legion 5i) [14] running Windows 10 as the edge server, and 80 000 ms as the application deadline. We use 99.0 as the penalty whenever any action does not meet the application deadline.

On deploying the trained DRL model on the Nano 2 GB, we obtained server response time as 4 ms and hence state as (4, 80 000). Passing this state to the trained DRL model estimated action values for the above nine actions were [34.99 94.11 104.69 47.84 52.89 51.61 42.16 44.99 41.32] with minimum value as 34.99, which corresponds to index 0 with the action (307 200, 30) and that becomes the best action for that state. The results for the best action and other actions are given in Table IV. As seen from the table, the best action (307 200, 30) meets the application deadline and has the lowest energy consumption of 33.0 mWh. The rest of the actions either miss the deadline and receive a penalty of 99.0 or meet the deadline but have a higher energy consumption than 33.0 mWh.

Similar to the Linux laptop experiments, we compared energy consumption from the proposed method for two deadlines 75 and 80 s with the Linux governors *ondemand* and *conservative*, and the comparison results are shown in Fig. 5.

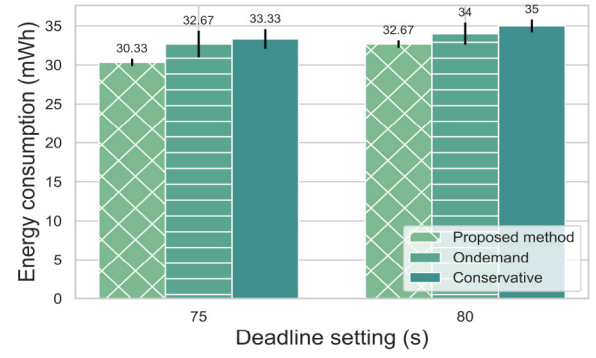


Fig. 5. Nano 2 GB energy consumption comparison.

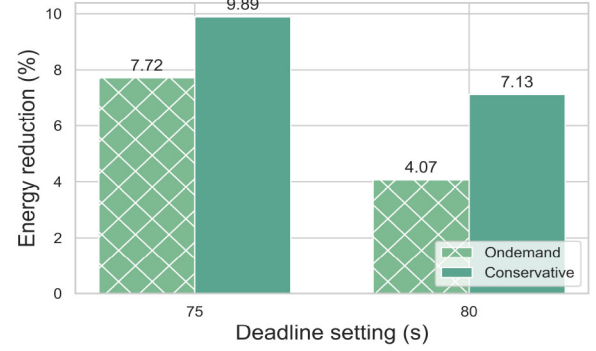


Fig. 6. Proposed method energy reduction compared to native governors on Nano 2 GB.

The *ondemand* governor consumes more energy in the range of 3.07%–7.72% while the *conservative* governor consumes more energy in the range of 7.13%–9.89% when compared to the proposed method energy consumption as shown in Fig. 6.

C. Raspberry Pi 4 Model B (RPi4B)

RPi4B [6] is the latest release of Raspberry Pi that is widely used as IoT device for various IoT applications. We use the 4 GB version of RPi4B running Raspberry Pi OS 64 bit beta version, which supports ten frequencies: 600 000, 700 000, 800 000, 900 000, 1 000 000, 1 100 000, 1 200 000, 1 300 000, 1 400 000, and 1 500 000 measured in unit kHz. Unlike the Linux laptop, the RPi4B does not have different voltage levels for different frequencies. The command *vcgencmd measure_volts core* returns the CPU core voltage as 0.8625 V for any of the above listed frequencies set as CPU frequency. *vcgencmd* is a command-line utility that can get various pieces of information from the Raspberry Pi including the CPU core voltage.

We selected three frequencies [600 000, 1 000 000, 1 500 000] for the local computation and data distribution ratios as 30, 50, and 70, which generates action space of nine actions [(600 000, 30), (600 000, 50), (600 000, 70), (1 000 000, 30), (1 000 000, 50), (1 000 000, 70), (1 500 000, 30), (1 500 000, 50), (1 500 000, 70)]. We used Legion 5i as the edge server and 60 000 ms as the application deadline. We use 99.0 as the penalty whenever any action does not meet the application deadline.

TABLE V
RPI4B TEST RESULTS FOR ALL ACTIONS WITH DEADLINE 60 000 MS

Action	Energy/Penalty	Deadline exceed?
(600000, 30)	53.0	No
(600000, 50)	56.0	No
(600000, 70)	99.0	Yes
(1000000, 30)	53.0	No
(1000000, 50)	55.0	No
(1000000, 70)	59.0	No
(1500000, 30)	54.0	No
(1500000, 50)	56.0	No
(1500000, 70)	58.0	No

On testing, we observed the response time as 4 ms and hence the state as (4, 60 000). Passing this state to the trained DRL model, the estimated action values for the above 9 actions were [56.91 55.11 104.98 52.29 60.04 57.99 53.32 58.46 64.20] with minimum value as 52.29, which corresponds to index 3 with the action (1 000 000, 30) and that becomes the best action for that state. The results for all the actions are given in Table V. As seen from the table, the best action (1 000 000, 30) meets the application deadline and has the lowest energy consumption of 53.0 mWh. The rest of the actions either miss the deadline and receive a penalty of 99.0 or meet the deadline but have the same or higher energy consumption than 53.0 mWh.

We compared the results of our proposed method with the native *ondemand* and *conservative* governors but the energy gain was negligible in the range of 1% as on Raspberry Pi, the CPU core voltage is the same for all the operating frequencies. The same CPU core voltage for all operating frequencies is also reflected in the results shown in Table V as the energy consumption for the same data distribution with different frequencies is more or less the same.

VI. CONCLUSION

To enable the IoT application running on the IoT device to meet the application time constraint and minimize the energy consumption, this article proposes a DRL scheme that performs decision making of: 1) *DVFS*: computation speed control on the device and 2) *Offloading*: communication from IoT device to edge servers. The proposed scheme learns the optimal data distribution policies and local computation DVFS frequency scaling by interacting with the system environment and learning the device, the network, and edge servers' behavior. The proposed scheme was tested on multiple EC environments with different IoT devices. Experimental results from various IoT devices using one edge server show that the proposed scheme always selects the optimal data distribution and the optimal DVFS frequency scaling that meets the IoT application deadline and minimizes the IoT device energy consumption. The trained DRL model estimates the correct action values even for some variations in the environmental situation, which it had not seen during training. The proposed scheme has significant energy savings when compared with the native Linux governors and will work better by selecting lower frequency if the IoT device supports lower

voltage levels for lower frequencies. The proposed method can be extended to find optimal data distribution with multiple edge servers available in the system by adding data distribution ratios for individual edge servers as action parameters such that the sum of the IoT edge and edge servers data distributions of those actions adds up to 100. Similarly, the IoT device will add response times from each edge server to the state parameters.

REFERENCES

- [1] D. Alshukri, R. V. Lavanya, E. P. Sumesh, and P. Krishnan, "Intelligent border security intrusion detection using IoT and embedded systems," in *Proc. 4th MEC Int. Conf. Big Data Smart City (ICBDSC)*, 2019, pp. 1–3.
- [2] H. Aydin, R. Melhem, D. Mosse, and P. Mejia-Alvarez, "Power-aware scheduling for periodic real-time tasks," *IEEE Trans. Comput.*, vol. 53, no. 5, pp. 584–600, May 2004.
- [3] D. Biswas V. Balagopal, R. Shafik, B. M. Al-Hashimi, and G. V. Merrett, "Machine learning for run-time energy optimisation in many-core systems," in *Design, Autom. & Test Europe Conf. & Exhibition (DATE)*, Lausanne, Switzerland, Mar. 2017.
- [4] "Jetson Nano 2GB Developer Kit." Nvidia. [Online]. Available: <https://developer.nvidia.com/embedded/jetson-nano-2gb-developer-kit> (accessed Sep. 10, 2021).
- [5] "The Linux Kernel Power Capping Framework." Linux Kernel Documentation. [Online]. Available: <https://www.kernel.org/doc/html/latest/power/powercap/powercap.html> (accessed Sep. 10, 2021).
- [6] "Raspberry Pi 4 Model B Specifications." Raspberry Pi Foundation. [Online]. Available: <https://www.raspberrypi.org/products/raspberry-pi-4-model-b/specifications/> (accessed Sep. 10, 2021).
- [7] V. Georgiev, "C library for X86 CPU Detection and Feature Extraction." [Online]. Available: <https://github.com/anrieff/libcpuid> (accessed Sep. 10, 2021).
- [8] S. Gong, Y. Xie, J. Xu, D. Niyato, and Y.-C. Liang, "Deep reinforcement learning for backscatter-aided data offloading in mobile edge computing," *IEEE Netw.*, vol. 34, no. 5, pp. 106–113, Sep./Oct. 2020.
- [9] H. Huang, Q. Ye, and H. Du, "Reinforcement learning based offloading for real-time applications in mobile edge computing," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2020, pp. 1–6.
- [10] H. Huang, M. Lin, L. T. Yang, and Q. Zhang, "Autonomous power management with double-Q reinforcement learning method," *IEEE Trans. Ind. Informat.*, vol. 16, no. 3, pp. 1938–1946, Mar. 2020.
- [11] "Intel Core i3-6006U Processor Specification." Intel. [Online]. Available: <https://ark.intel.com/content/www/us/en/ark/products/91157/intel-core-i3-6006u-processor-3m-cache-2-00-ghz.html> (accessed Sep. 10, 2021).
- [12] F. M. M. Ul Islam and M. Lin, "Hybrid DVFS scheduling for real-time systems based on reinforcement learning," *IEEE Syst. J.*, vol. 11, no. 2, pp. 931–940, Jun. 2017.
- [13] B. Jang, M. Kim, G. Harerimana, and J. W. Kim, "Q-learning algorithms: A comprehensive classification and applications," *IEEE Access*, vol. 7, pp. 133653–133667, 2019.
- [14] "Lenovo Legion 5i (17") Gaming Laptop." Lenovo. [Online]. Available: <https://www.lenovo.com/ca/en/laptops/legion-laptops/legion-5-series/Legion-5i-17/p/88GMY501441> (accessed Sep. 10, 2021).
- [15] H. Li, K. Ota, and M. Dong, "Learning IoT in edge: Deep learning for the Internet of Things with edge computing," *IEEE Netw.*, vol. 32, no. 1, pp. 96–101, Jan./Feb. 2018.
- [16] J. Li, H. Gao, T. Lv, and Y. Lu, "Deep reinforcement learning based computation offloading and resource allocation for MEC," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, 2018, pp. 1–6.
- [17] X. Lin, Y. Wang, Q. Xie, and M. Pedram, "Task scheduling with dynamic voltage and frequency scaling for energy minimization in the mobile cloud computing environment," *IEEE Trans. Services Comput.*, vol. 8, no. 2, pp. 175–186, Mar./Apr. 2015.
- [18] F. Liu, Z. Huang, and L. Wang, "Energy-efficient collaborative task computation offloading in cloud-assisted edge computing for IoT sensors," *Sensors*, vol. 19, no. 5, p. 1105, 2019.
- [19] G. liu. "UM25C Documents and Software." [Online]. Available: <https://www.mediafire.com/folder/q2b8h079hpywq/UM25> (accessed Sep. 10, 2021).

- [20] N. C. Luong *et al.*, "Applications of deep reinforcement learning in communications and networking: A survey," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 4, pp. 3133–3174, 4th Quart., 2019.
- [21] X. Ma *et al.*, "A survey on deep learning empowered IoT applications," *IEEE Access*, vol. 7, pp. 181721–181732, 2019.
- [22] T. Malche and P. Maheshwary, "Internet of Things (IoT) for building smart home system," in *Proc. Int. Conf. I-SMAC (IoT Social, Mobile, Analytics Cloud) (I-SMAC)*, 2017, pp. 65–70.
- [23] S. K. Panda and M. Lin, "IoT Device Energy Consumption Measurement Using a Machine Learning Model." [Online]. Available: <https://ahsn-committees.comsoc.org/files/2021/07/IOT-AHSN-Newsletter-June-2021-V2.pdf> (accessed Sep. 10, 2021).
- [24] E. I. Papageorgiou *et al.*, "Short survey of artificial intelligent technologies for defect detection in manufacturing," in *Proc. 12th Int. Conf. Inf. Intell. Syst. Appl. (IISA)*, 2021, pp. 1–7.
- [25] J.-G. Park, N. Dutt, and S.-S. Lim, "An interpretable machine learning model enhanced integrated CPU-GPU DVFS governor," *ACM Trans. Embedded Comput. Syst.*, vol. 20, no. 6, p. 108, 2021.
- [26] R. Porkodi and V. Bhuvaneswari, "The Internet of Things (IoT) applications and communication enabling technology standards: An overview," in *Proc. Int. Conf. Intell. Comput. Appl.*, 2014, pp. 324–329.
- [27] W. Qiang and Z. Zhongli, "Reinforcement learning model, algorithms and its application," in *Proc. Int. Conf. Mechatronic Sci. Electr. Eng. Comput. (MEC)*, 2011, pp. 1143–1146.
- [28] K. Rambabu, V. Haritha, S. N. Srinivas, and P. S. Reddy, "IoT based human intrusion detection system using lab view," *Int. J. Innov. Technol. Exploring Eng.*, vol. 8, no. 6S4, pp. 557–560, Apr. 2019.
- [29] D. Ramegowda and M. Lin, "Can learning-based hybrid DVFS technique adapt to different Linux embedded platforms?" in *Proc. IEEE Ubiquitous Intell. Comput. (UIC)*, 2021, pp. 170–177.
- [30] Y. Ren, Y. Sun, and M. Peng, "Deep reinforcement learning based computation offloading in fog enabled Industrial Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 17, no. 7, pp. 4978–4987, Jul. 2021.
- [31] W. Shi, G. Pallis, and Z. Xu, "Edge computing [scanning the issue]," *Proc. IEEE*, vol. 107, no. 8, pp. 1474–1481, Aug. 2019.
- [32] "Mnist Database." Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/MNIST_database (accessed Sep. 10, 2021).
- [33] "Weblate (Bot) Xorg/CPU-X Free Software." [Online]. Available: <https://github.com/XOrg/CPU-X> (accessed Sep. 10, 2021).
- [34] Q. Zhang, M. Lin, L. T. Yang, Z. Chen, S. U. Khan, and P. Li, "A double deep Q -learning model for energy-efficient edge scheduling," *IEEE Trans. Services Comput.*, vol. 12, no. 5, pp. 739–749, Sep./Oct. 2019.
- [35] X. Zhang, Y. Shen, B. Yang, W. Zang, and S. Wang, "DRL based data offloading for intelligent reflecting surface aided mobile edge computing," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, 2021, pp. 1–7.
- [36] T. Zhou and M. Lin, "Deadline-aware deep-recurrent- Q -network governor for smart energy saving," *IEEE Trans. Netw. Sci. Eng.*, early access, Oct. 27, 2021, doi: [10.1109/TNSE.2021.3123280](https://doi.org/10.1109/TNSE.2021.3123280).



Saroj Kumar Panda is currently pursuing the Master of Science degree in computer science with St. Francis Xavier University, Antigonish, NS, Canada.

His research interests include machine learning, deep learning, edge computing, and Internet of Things.



Man Lin (Senior Member, IEEE) received the B.E. degree in computer science and technology from Tsinghua University, Beijing, China, in 1994, and the Ph.D. degree from Linköping University, Linköping, Sweden, in 2000.

She is currently a Professor of Computer Science with St. Francis Xavier University, Antigonish, NS, Canada. Her research interests include cyber-physical systems, real-time scheduling, power-aware computing, machine learning, system analysis, and optimization algorithms.



Ti Zhou is currently pursuing the Master of Science degree in computer science with St. Francis Xavier University, Antigonish, NS, Canada.

His research interests include self-optimizing system design, data-driven system design, and system-level power management.